REPORT

Title

File System Search Optimization Using Binary Search

Aim

To analyze how Binary Search improves file retrieval efficiency in a sorted file directory and understand its divide-and-conquer approach, requirements, time complexity, and advantages over linear searching.

Problem Statement

An operating system maintains sorted file directories for quick access. Analyze how Binary Search improves file retrieval efficiency.

The solution should:

- Identify the requirements for Binary Search.
- Explain the divide-and-conquer principle.
- Analyze its time complexity.
- Justify why Binary Search is suitable for efficient file retrieval.

Introduction

An operating system may maintain a large number of files in directories. Searching sequentially through every file can become inefficient when the number of files is very large.

Binary Search provides a faster searching technique when the file names are maintained in a **sorted order**.

Instead of checking every file, Binary Search examines the middle file and eliminates approximately half of the remaining search space after every comparison.

Requirements

Binary Search requires the following:

1. Sorted Index

The file names must be arranged in sorted order.

Example:

Algorithms.pdf

Database.docx

Networks.pdf

OperatingSystem.pdf

Programming.c

ResearchPaper.pdf

SystemDesign.docx

WebDevelopment.html

2. Comparable File Names

The requested file name must be compared with the file name at the middle position.

3. Efficient Access

Binary Search is especially suitable when the directory provides efficient access to the middle element, such as an array-based index.

Working of Binary Search

Suppose the system wants to find:

ResearchPaper.pdf

The algorithm initially considers the entire sorted list.

Step 1

Check the middle element.

OperatingSystem.pdf

Since:

ResearchPaper.pdf > OperatingSystem.pdf

the left half can be discarded.

Step 2

Search the remaining right half.

The middle element is checked again.

If the target is greater than the middle element, search continues in the right half; otherwise, the left half is selected.

Step 3

The search range continues to become smaller until:

- The target file is found, or
- The search range becomes empty.

Divide and Conquer Principle

Binary Search follows the **Divide and Conquer** approach.

Divide

Divide the current search range into two halves.

Conquer

Determine which half can contain the target based on comparison.

Repeat

Continue searching only in the selected half.

Thus, approximately half of the remaining files are eliminated after each comparison.

Algorithm

1. Set low = 0.
2. Set high = n - 1.
3. Calculate the middle position.
4. Compare the target file name with the middle file name.
5. If they are equal, return the file position.

6. If the target comes after the middle file, search the right half.
7. Otherwise, search the left half.
8. Repeat until the file is found or low > high.
9. If the search range becomes empty, report that the file was not found.

Source Code

The program implements Binary Search on a sorted list of file names.

Source file:

SOURCE CODE AT2 CO3.c

The strcmp() function is used to compare file names.

Sample Output

File System Search using Binary Search

==========

Available Files:

1. Algorithms.pdf
2. Database.docx
3. Networks.pdf
4. OperatingSystem.pdf
5. Programming.c
6. ResearchPaper.pdf
7. SystemDesign.docx
8. WebDevelopment.html

Enter file name to search: ResearchPaper.pdf

File found!

File: ResearchPaper.pdf

Index: 5

Time Complexity Analysis

Let:

At every step, Binary Search eliminates approximately half of the remaining elements.

Therefore:

The number of divisions required is:

Therefore, the time complexity is:

Best Case

The target is found at the first middle comparison.

Average Case

Worst Case

Space Complexity

The implementation uses an iterative approach, so it requires constant additional memory.

Comparison with Linear Search

Feature	Linear Search	Binary Search
Data requirement	No sorting required	Must be sorted
Best Case	O(1)	O(1)
Average Case	O(n)	O(log n)
Worst Case	O(n)	O(log n)
Search method	Sequential	Divide and conquer
Large datasets	Less efficient	More efficient

Why Binary Search is Suitable

Binary Search is suitable for large sorted file directories because it significantly reduces the number of comparisons.

For example, with approximately **1,000,000 files**:

Linear Search

May require up to approximately:

comparisons.

Binary Search

Requires approximately:

comparisons in the worst case, ignoring the cost of string comparison itself.

This demonstrates the major efficiency advantage of Binary Search.

Advantages

- Very fast for large sorted datasets.
- Uses divide-and-conquer.
- Reduces the search space by half at each step.
- Requires only constant extra space in the iterative implementation.
- Suitable for indexed file directories.

Limitations

- The file directory must be sorted.
- Maintaining sorted order can require additional work when files are added or removed.
- Binary Search is not ideal for data structures where accessing the middle element is expensive.

Applications

Binary Search can be used for:

- File directory searching
- Database indexes
- Dictionary lookup
- Searching sorted records
- Library catalog systems
- Contact directories
- Indexed storage systems

Result

Binary Search was successfully applied to a sorted file directory. The divide-and-conquer approach reduces the search space by half after each comparison.

The algorithm achieves:

average and worst-case search time, compared with:

for Linear Search.

Therefore, Binary Search provides a highly efficient solution for retrieving files from **large sorted directories**.